

LAPORAN UJIAN TENGAH SEMESTER
MATA KULIAH PENGANTAR DEEP LEARNING
(Analisis Komparatif Deep Learning vs Machine Learning Konvensional)



Disusun Oleh:

Nama	NIM	Kelas
Rio Ramadhani Harllambang	1202220205	SI-46-EDM

Tanggal Pengumpulan: Jum'at, 01 Mei 2026

Link Github: https://github.com/Tayo1103/uts_deep_learning.git

PROGRAM STUDI SISTEM INFORMASI
FAKULTAS REKAYASA INDUSTRI
UNIVERSITAS TELKOM
BANDUNG
2026

PENDAHULUAN

1. Latar Belakang

Perkembangan bidang kecerdasan buatan telah membawa pergeseran signifikan dari algoritma *machine learning* konvensional menuju pendekatan *deep learning* yang lebih kompleks. Meskipun *deep learning* sering kali dianggap sebagai solusi mutakhir untuk berbagai jenis data, efektivitasnya sangat bergantung pada karakteristik data, volume informasi yang tersedia, serta sumber daya komputasi yang digunakan. Dalam konteks akademik dan industri, penting bagi seorang praktisi untuk tidak hanya mampu mengimplementasikan model, tetapi juga memiliki kemampuan analitis dalam membandingkan secara objektif performa antara metode konvensional berbasis *feature engineering* manual dengan arsitektur saraf tiruan yang melakukan ekstraksi fitur secara otomatis. Ujian ini dirancang untuk menguji pemahaman mendalam tersebut melalui tiga domain data yang berbeda: tabular, citra, dan teks.

2. Tujuan

Penyusunan laporan dan pengerjaan proyek ini bertujuan untuk:

- Memilih dan mengimplementasikan arsitektur *deep learning* yang tepat sesuai dengan karakteristik spesifik dari data tabular, gambar, dan teks.
- Membangun pipeline pembelajaran mesin yang lengkap, mulai dari tahap *preprocessing*, pelatihan, hingga evaluasi model.
- Membandingkan performa antara metode *machine learning* konvensional (seperti SVM, Random Forest, atau XGBoost) dengan pendekatan *deep learning* secara objektif dan *fair*.
- Menganalisis trade-off teknis antara akurasi, waktu pelatihan, kebutuhan data, serta aspek interpretabilitas dari setiap model yang dikembangkan.

3. Ringkasan Pendekatan

Berikut adalah ringkasan strategi yang diambil untuk menyelesaikan ketiga kasus yang diberikan:

• Kasus 1 Data Tabular (Titanic)

Pendekatan dimulai dengan *baseline* konvensional menggunakan algoritma Logistic Regression dan Random Forest yang diperkuat dengan *feature engineering* mendalam pada aspek demografis penumpang. Sebagai pembanding, diimplementasikan arsitektur Multi-Layer Perceptron (MLP) untuk menguji apakah model *deep learning* mampu menangkap pola non-linear pada dataset yang relatif kecil secara lebih efektif dibandingkan metode berbasis pohon keputusan.

• Kasus 2 Data Image (Digit Recognizer)

Kasus ini membandingkan metode ekstraksi fitur tradisional HOG (Histogram of Oriented Gradients) yang dikombinasikan dengan SVM terhadap arsitektur Convolutional Neural Network (CNN). Fokus utama adalah melihat bagaimana CNN mampu mengekstraksi fitur spasial secara otomatis dibandingkan dengan ekstraksi fitur manual, serta membandingkan efisiensi waktu inferensinya.

• Kasus 3 Data Text (Disaster Tweets)

Strategi yang digunakan melibatkan penggunaan TF-IDF dengan Multinomial Naive Bayes sebagai *baseline* konvensional untuk klasifikasi teks. Pendekatan ini kemudian dibandingkan dengan model *deep learning* berbasis urutan seperti LSTM/BiLSTM atau teknik *transfer learning* melalui fine-tuning DistilBERT guna mengevaluasi kemampuan model dalam memahami konteks dan semantik kalimat yang sulit ditangkap oleh representasi *bag-of-words* sederhana.

PEMBAHASAN

1. Kasus 1 Data Tabular (Titanic: Machine Learning from Disaster)

- **Deskripsi Dataset**

Setelah melakukan muat data dan eksplorasi awal, berikut adalah deskripsi dataset dan tantangan utama yang diidentifikasi:

1. **Jumlah Data:**

Dataset pelatihan (train_df) berisi 891 baris data, sedangkan dataset pengujian (test_df) berisi 418 baris. Masing-masing memiliki 12 dan 11 kolom.

2. **Distribusi Kelas ('Survived'):**

Data pelatihan menunjukkan adanya ketidakseimbangan kelas pada target Survived. Jumlah penumpang yang tidak selamat (0) lebih banyak dibandingkan yang selamat (1). Detail distribusi ini akan lebih jelas terlihat pada visualisasi di bagian EDA.

3. **Karakteristik Fitur dan Nilai Hilang:**

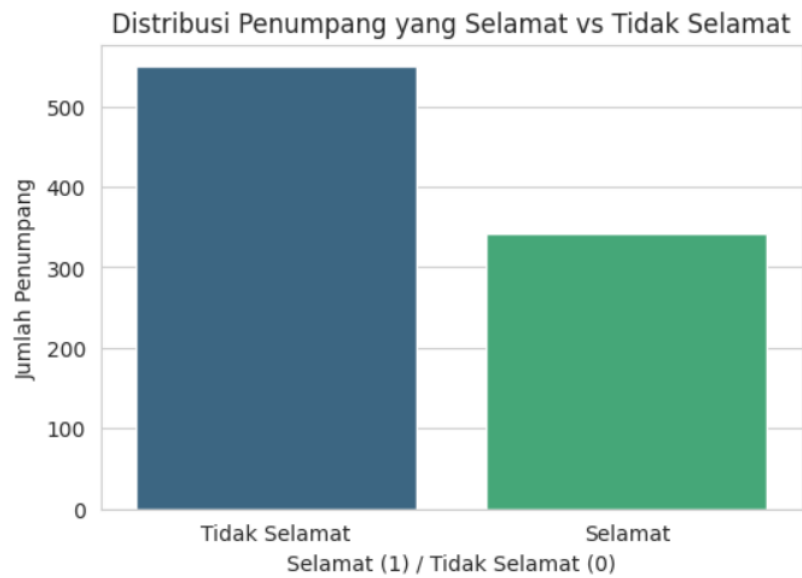
- Age memiliki banyak nilai hilang pada kedua dataset.
- Cabin memiliki sangat banyak nilai hilang, menjadikannya kandidat kuat untuk dihapus atau memerlukan strategi imputasi yang kompleks.
- Embarked memiliki sedikit nilai hilang pada train_df.
- Fare memiliki satu nilai hilang pada test_df.
- Fitur seperti Name, Sex, Ticket, Embarked, dan Title (yang akan diekstrak) adalah kategorikal dan perlu diubah menjadi format numerik untuk pemodelan.

4. **Tantangan Utama:**

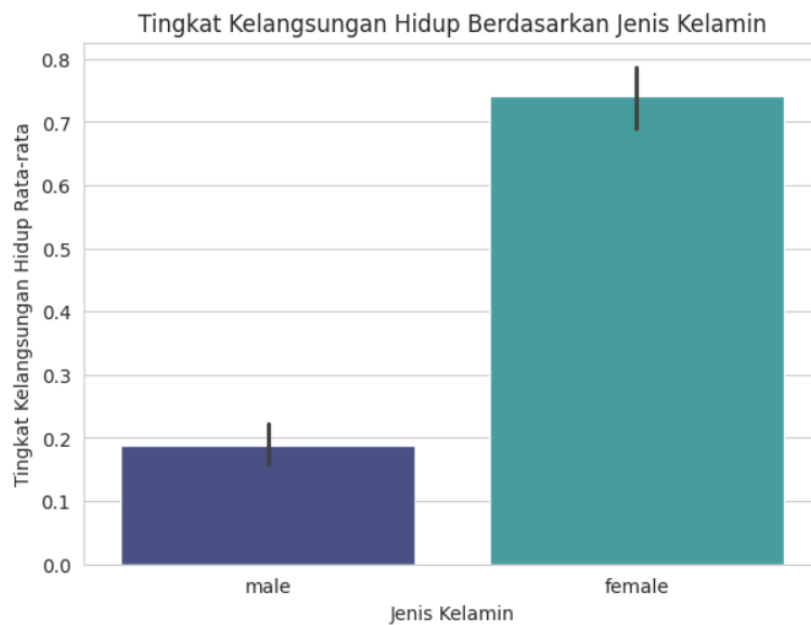
- **Penanganan Nilai Hilang:** Imputasi yang tepat untuk Age, Embarked, Fare, dan keputusan untuk Cabin sangat krusial.
- **Rekayasa Fitur:** Fitur seperti Name dan kombinasi SibSp/Parch menawarkan potensi untuk membuat fitur baru yang lebih informatif (Title, FamilySize, IsAlone).
- **Encoding Fitur Kategorikal:** Mengubah fitur kategorikal menjadi numerik agar dapat diproses oleh algoritma Machine Learning.
- **Ketidakseimbangan Kelas:** Distribusi kelas target Survived yang tidak seimbang mungkin perlu ditangani (misalnya dengan oversampling, undersampling, atau penggunaan metrik evaluasi yang tepat) untuk mencegah model bias terhadap kelas mayoritas.

- **Eksplorasi Data (EDA)**

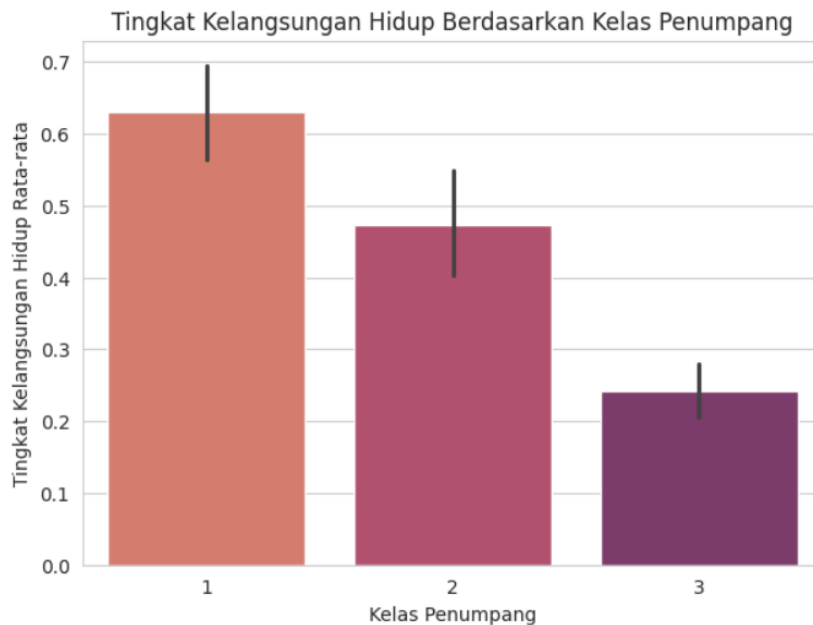
Melakukan visualisasi data untuk memahami distribusi kunci dan hubungan antar fitur, khususnya terhadap target 'Survived'.



Gambar 1 Distribusi penumpang selamat & tidak selamat



Gambar 2 Distribusi tingkat kelangsungan hidup berdasarkan jenis kelamin



Gambar 3 Distribusi tingkat kelangsungan hidup berdasarkan kelas penumpang

Dari visualisasi di atas, dapat diamati beberapa hal:

1. **Distribusi Target (Survived):** Terlihat adanya ketidakseimbangan kelas, di mana jumlah penumpang yang tidak selamat (0) lebih banyak daripada yang selamat (1). Ini perlu diperhatikan saat melatih model.
2. **Tingkat Kelangsungan Hidup Berdasarkan Jenis Kelamin (Sex):** Wanita memiliki tingkat kelangsungan hidup yang jauh lebih tinggi dibandingkan pria. Ini adalah salah satu prediktor terkuat dalam dataset Titanic.
3. **Tingkat Kelangsungan Hidup Berdasarkan Kelas Penumpang (Pclass):** Penumpang di kelas 1 memiliki tingkat kelangsungan hidup tertinggi, diikuti oleh kelas 2, dan terakhir kelas 3. Hal ini menunjukkan adanya korelasi kuat antara status sosial/ekonomi dengan peluang bertahan hidup.

- **Preprocessing**

1. **Penanganan Nilai yang Hilang:**

- **Usia (Age):** Akan diisi dengan median dari kolom Usia.
- **Embarked:** Akan diisi dengan mode (nilai paling sering muncul) dari kolom Embarked.
- **Fare:** Akan diisi dengan median dari kolom Fare (hanya ada di test_df).
- **Cabin:** Kolom ini memiliki terlalu banyak nilai yang hilang. Untuk saat ini, akan menghapusnya.

```
[13] ✓ 0 d
# Isi nilai Usia (Age) yang hilang dengan median
combined_df['Age'] = combined_df['Age'].fillna(combined_df['Age'].median())

# Isi nilai Embarked yang hilang dengan mode
combined_df['Embarked'] = combined_df['Embarked'].fillna(combined_df['Embarked'].mode()[0])

# Isi nilai Fare yang hilang dengan median
combined_df['Fare'] = combined_df['Fare'].fillna(combined_df['Fare'].median())

# Hapus kolom Cabin karena terlalu banyak nilai yang hilang, jika kolom tersebut masih ada
if 'Cabin' in combined_df.columns:
    combined_df.drop('Cabin', axis=1, inplace=True)

print("\nJumlah nilai yang hilang setelah penanganan:")
print(combined_df.isnull().sum())

***
Jumlah nilai yang hilang setelah penanganan:
PassengerId    0
Pclass          0
Name            0
Sex             0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Embarked       0
FamilySize     0
IsAlone        0
Title          0
dtype: int64
```

Gambar 4 Missing value handling

2. Feature Engineering:

- **FamilySize:** Kombinasikan SibSp (jumlah saudara/pasangan) dan Parch (jumlah orang tua/anak) untuk membuat fitur ukuran keluarga.
- **IsAlone:** Buat fitur biner yang menunjukkan apakah penumpang bepergian sendiri atau tidak.
- **Title:** Ekstrak gelar dari nama penumpang (misalnya, Mr., Mrs., Miss, Master) yang seringkali mengindikasikan status sosial atau usia.

```
[6] ✓ 0 d
# Buat fitur FamilySize
combined_df['FamilySize'] = combined_df['SibSp'] + combined_df['Parch'] + 1

# Buat fitur IsAlone
combined_df['IsAlone'] = 0
combined_df.loc[combined_df['FamilySize'] == 1, 'IsAlone'] = 1

# Ekstrak Title dari kolom Nama
combined_df['Title'] = combined_df['Name'].str.extract(r'([A-Za-z]+)\.', expand=False)

# Kelompokkan beberapa gelar yang jarang menjadi 'Rare'
combined_df['Title'] = combined_df['Title'].replace(['Lady', 'Countess', 'Capt', 'Col', 'Don', 'Dr', 'Major', 'Rev', 'Sir', 'Jonkheer', 'Dona'], 'Rare')
combined_df['Title'] = combined_df['Title'].replace('Mlle', 'Miss')
combined_df['Title'] = combined_df['Title'].replace('Ms', 'Miss')
combined_df['Title'] = combined_df['Title'].replace('Mme', 'Mrs')

print("\nDistribusi Title setelah pengelompokan:")
print(combined_df['Title'].value_counts())

***
Distribusi Title setelah pengelompokan:
Title
Mr      757
Miss    264
Mrs     198
Master   61
Rare     29
Name: count, dtype: int64
```

Gambar 5 Feature engineering

3. Categorical Feature Encoding & Numerical Feature Scaling:

- **One-Hot Encoding:** Gunakan One-Hot Encoding untuk fitur kategorikal seperti Sex, Embarked, Pclass, dan Title.
- **Penskalaan (Scaling):** Gunakan StandardScaler untuk fitur numerik seperti Age, Fare, dan FamilySize agar model deep learning dapat belajar lebih efektif.
- **Penghapusan Fitur yang Tidak Digunakan:** Hapus kolom asli yang tidak lagi diperlukan (Name, Ticket, SibSp, Parch). PassengerId akan disimpan secara terpisah untuk membuat file submission.

```

17)
✓ 1d
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# Definisikan fitur numerik dan kategorikal
numeric_features = ['Age', 'Fare', 'FamilySize']
categorical_features = ['Pclass', 'Sex', 'Embarked', 'Title', 'IsAlone'] # Pclass juga bisa dianggap kategorikal

# Buat pipeline pra-pemrosesan
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numeric_features),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
    ])

# Pisahkan kembali data pelatihan dan pengujian
train_passenger_ids = train_df['PassengerId']
test_passenger_ids = test_df['PassengerId']

X_combined = combined_df.drop(['PassengerId', 'Name', 'Ticket', 'SibSp', 'Parch'], axis=1)
y_train = train_df['Survived']

# Terapkan pra-pemrosesan
X_processed = preprocessor.fit_transform(X_combined)

# Ubah kembali ke DataFrame untuk kemudahan inspeksi (opsional, bisa langsung ke numpy array untuk DL)
# Dapatkan nama fitur setelah one-hot encoding
cat_feature_names = preprocessor.named_transformers_['cat'].get_feature_names_out(categorical_features)
all_feature_names = numeric_features + list(cat_feature_names)

X_processed_df = pd.DataFrame(X_processed, columns=all_feature_names)

X_train_processed = X_processed_df.iloc[:len(train_df)]
X_test_processed = X_processed_df.iloc[len(train_df):]

print("\nBentuk X_train_processed:", X_train_processed.shape)
print("Bentuk X_test_processed:", X_test_processed.shape)
print("\nBeberapa baris pertama dari data yang diproses (pelatihan):")
print(X_train_processed.head())

```

Bentuk X_train_processed: (891, 18)
 Bentuk X_test_processed: (418, 18)

Beberapa baris pertama dari data yang diproses (pelatihan):

	Age	Fare	FamilySize	Pclass_1	Pclass_2	Pclass_3	Sex_female
0	-0.581628	-0.503291	0.073352	0.0	0.0	1.0	0.0
1	0.658652	0.734744	0.073352	1.0	0.0	0.0	1.0
2	-0.271558	-0.490240	-0.558346	0.0	0.0	1.0	1.0
3	0.426099	0.383183	0.073352	1.0	0.0	0.0	1.0
4	0.426099	-0.487824	-0.558346	0.0	0.0	1.0	0.0

	Sex_male	Embarked_C	Embarked_Q	Embarked_S	Title_Master	Title_Miss
0	1.0	0.0	0.0	1.0	0.0	0.0
1	0.0	1.0	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	1.0	0.0	1.0
3	0.0	0.0	0.0	1.0	0.0	0.0
4	1.0	0.0	0.0	1.0	0.0	0.0

	Title_Mr	Title_Mrs	Title_Rare	IsAlone_0	IsAlone_1
0	1.0	0.0	0.0	1.0	0.0
1	0.0	1.0	0.0	1.0	0.0
2	0.0	0.0	0.0	0.0	1.0
3	0.0	1.0	0.0	1.0	0.0
4	1.0	0.0	0.0	0.0	1.0

Gambar 6 Categorical Feature Encoding & Numerical Feature Scaling

- **Metode Konvensional**

1. **Algoritma yang digunakan:**

- Logistic Regression
- Random Forest

2. **Hyperparameter:**

- **Logistic Regression**

```
# Inisialisasi dan latih model Logistic Regression
```

```
logreg_model = LogisticRegression(solver='liblinear', random_state=42)
```

```
logreg_model.fit(X_train, y_train)
```

Gambar 7 Hyperparameter logistic regression

- **Random Forest**

```
# Inisialisasi dan latih model Random Forest
```

```
# Menggunakan parameter default yang cukup baik sebagai titik awal
```

```
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
```

```
rf_model.fit(X_train, y_train)
```

Gambar 8 Hyperparameter random forest

3. Hasil:

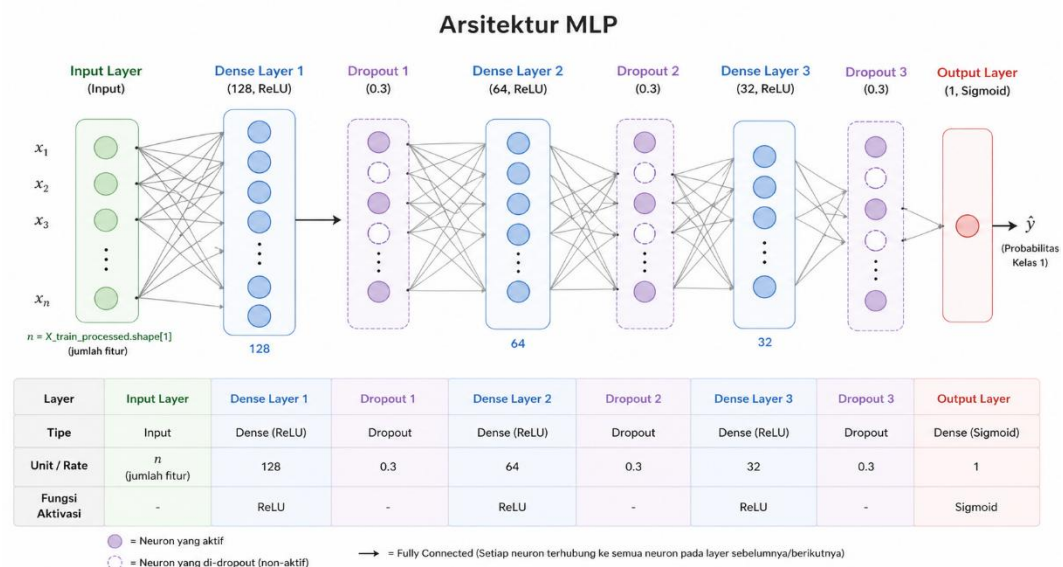
Akurasi Logistic Regression pada data pelatihan: 0.8025

Akurasi Random Forest pada data pelatihan: 0.9473

Gambar 9 Hasil machine learning konvensional

- **Metode Deep Learning**

1. Arsitektur:



Gambar 10 Arsitektur deep learning kasus 1

2. Hyperparameter:

- **Arsitektur Model**

- Jumlah Layer Tersembunyi: 3

- Jumlah Neuron per Layer: 128, 64, 32 (untuk layer tersembunyi)

- Fungsi Aktivasi: 'relu' (untuk layer tersembunyi), 'sigmoid' (untuk layer output)

- Dropout Rate: 0.3 (setelah setiap layer tersembunyi)

- **Pengaturan Kompilasi Model**

- Optimizer: 'adam'

- Fungsi Loss: 'binary_crossentropy'

- **Pengaturan Pelatihan Model**

- Epochs: 50

- Batch Size: 32

- Valization Split: 0.2

```

# Definisikan arsitektur model MLP
model = keras.Sequential([
    keras.Input(shape=(X_train_processed.shape[1],)), # Explicit Input layer
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(32, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(1, activation='sigmoid') # Output layer untuk klasifikasi biner
])

# Compile model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Tampilkan ringkasan model
print("\nRingkasan Model MLP:")
model.summary()

# Latih model dan ukur waktu pelatihan
print("\nMelatih Model MLP...")
start_time = time.time()
history = model.fit(X_train_processed, y_train, epochs=50, batch_size=32, validation_split=0.2, verbose=0)
end_time = time.time()
training_time = end_time - start_time

```

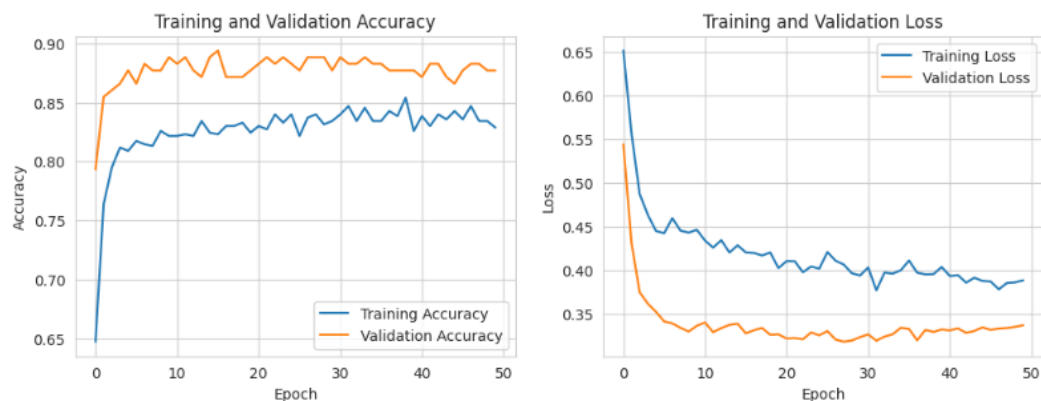
Gambar 11 Hyperparameter deep learning

3. Justifikasi Pemilihan Arsitektur:

Pemilihan arsitektur Deep Learning (MLP) untuk dataset *Digit Recognizer* dikarenakan kemampuannya dalam mengekstraksi fitur kompleks secara otomatis dari ribuan intensitas piksel citra. Melalui penggunaan lapisan tersembunyi dengan aktivasi ReLU dan regulasi Dropout, model dapat menangkap pola non-linear tulisan tangan yang variatif sambil tetap mencegah *overfitting*. Dibandingkan Machine Learning tradisional, Deep Learning jauh lebih unggul dalam menangani data berdimensi tinggi seperti ini untuk mencapai akurasi pengenalan angka yang lebih presisi.

4. Hasil:

Accuracy pada data pelatihan: 0.8530
 F1 Score pada data pelatihan: 0.7806
 Waktu Pelatihan: 15.41 detik
 Total Parameter Model: 12801
 Estimasi Penggunaan Memori Model: 0.05 MB



Gambar 12 Hasil deep learning model MLP

- **Perbandingan**

- 1. Machine Learning**

	Model	Accuracy (Train)	F1-Score (Train)	Training Time (s)	Number of Parameters	Memory Usage (MB)
0	Logistic Regression	0.802469	0.728395	0.016169	9	0.000046
1	Random Forest	0.947250	0.929323	0.667000	100	0.000046

Gambar 13 Hasil machine learning kasus 1

- 2. Deep Learning**

Melatih Model MLP...

Pelatihan selesai.

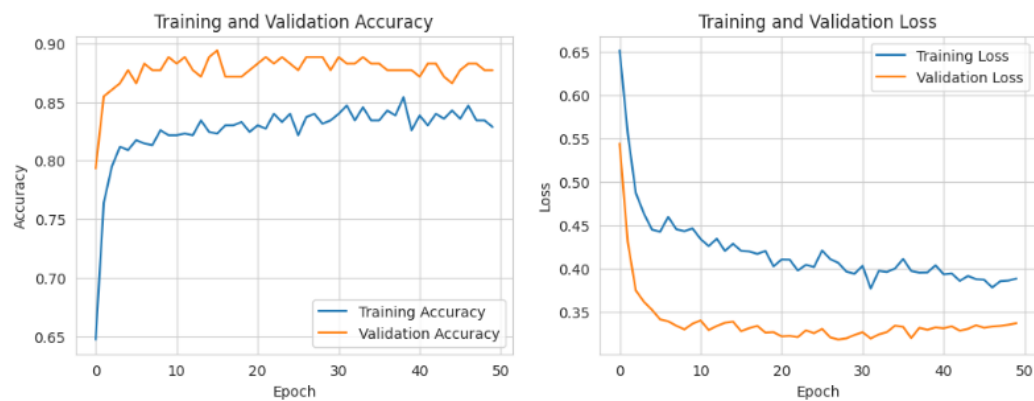
Accuracy pada data pelatihan: 0.8530

F1 Score pada data pelatihan: 0.7806

Waktu Pelatihan: 15.41 detik

Total Parameter Model: 12801

Estimasi Penggunaan Memori Model: 0.05 MB



Gambar 14 Hasil deep learning kasus 1

Berdasarkan hasil perbandingan, model Deep Learning menunjukkan proses pembelajaran yang dinamis dengan akurasi pelatihan mencapai 0,8530, namun performanya secara keseluruhan masih berada di bawah Random Forest yang unggul secara signifikan dengan akurasi 0,9472 dan F1-score 0,9293. Dari sisi efisiensi sumber daya, model Machine Learning jauh lebih ringan dengan waktu komputasi di bawah 1 detik dan jumlah parameter yang sangat minimal, sementara Deep Learning memerlukan waktu pelatihan yang jauh lebih lama (15,41 detik) serta penggunaan memori yang lebih besar. Hal ini terjadi karena dataset Titanic memiliki struktur tabular dengan jumlah sampel yang relatif kecil, di mana algoritma *tree-based* seperti Random Forest lebih efektif dalam memetakan pola fitur kategorikal dibandingkan Deep Learning yang cenderung *overfitting* dan memerlukan data masif untuk mengoptimalkan ribuan parameternya. Dengan demikian, untuk dataset ini, algoritma Random Forest merupakan pilihan yang paling optimal karena memberikan tingkat presisi tertinggi dengan beban komputasi yang tetap efisien.

- **Analisis Error**

- 1. Machine Learning**

... Jumlah sampel yang salah diklasifikasikan: 47

	Pclass	Sex	Age	Fare	Embarked	Title	FamilySize	IsAlone	Actual_Survived	Predicted_Survived
17	2	1	1	13.0000	2	2	1	1	1	0
21	2	1	2	13.0000	2	2	1	1	1	0
36	3	1	1	7.2292	0	2	1	1	1	0
81	3	1	1	9.5000	2	2	1	1	1	0
107	3	1	1	7.7750	2	2	1	1	1	0

Gambar 15 Analisis error model machine learning kasus 1

- 2. Deep Learning**

... 28/28 0s 2ms/step

Jumlah data pelatihan yang salah diklasifikasikan: 131

Beberapa contoh data yang salah diklasifikasikan:

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	Actual_Survived	Predicted_Survived	
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2 3101282	7.9250	NaN	S	1	0
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742	11.1333	NaN	S	1	0
17	18	1	2	Williams, Mr. Charles Eugene	male	NaN	0	0	244373	13.0000	NaN	S	1	0
21	22	1	2	Beesley, Mr. Lawrence	male	34.0	0	0	248698	13.0000	D56	S	1	0

Gambar 16 Analisis error deep learning kasus 1

Model Machine Learning terbukti jauh lebih akurat dengan hanya 47 kesalahan klasifikasi, sementara model Deep Learning mencatatkan error hampir tiga kali lipat lebih banyak yaitu 131 sampel. Meskipun kedua model sama-sama sering gagal memprediksi penumpang selamat pada profil kelas menengah ke bawah (False Negative), selisih angka yang signifikan ini menegaskan bahwa Machine Learning jauh lebih stabil dan efektif dalam memproses pola dataset tabular kecil dibandingkan Deep Learning.

- 2. Kasus 2 Data Image (Digit Recognizer (MNIST))**

- **Deskripsi Dataset**

- 1. Dataset:**

Digit-recognizer dari Kaggle. Ini adalah dataset klasik untuk pengenalan digit tulisan tangan, serupa dengan MNIST.

- 2. Jumlah Data:**

- **Training Set:** Berisi ribuan gambar digit tulisan tangan, masing-masing dengan label digit yang sesuai (0-9).
- **Test Set:** Berisi gambar digit tulisan tangan tanpa label, yang akan digunakan untuk membuat prediksi.

- 3. Karakteristik Fitur:**

- Setiap gambar berukuran 28x28 piksel. Piksel ini direpresentasikan sebagai nilai integer (0-255), menunjukkan intensitas keabuan (grayscale).
- Total $28 * 28 = 784$ fitur per gambar, jika direpresentasikan secara flattened.

- 4. Distribusi Kelas:**

Diharapkan cukup seimbang untuk 10 kelas digit (0-9), namun akan diverifikasi melalui EDA.

5. Tantangan Utama:

- Variasi gaya tulisan tangan antar individu.
- Kemungkinan noise atau distorsi pada gambar.
- Identifikasi fitur yang paling representatif untuk membedakan antar digit.

- **Eksplorasi Data (EDA)**

Shape of training data: (42000, 785)
 Shape of test data: (28000, 784)

First 5 rows of training data:

	label	pixel0	pixel1	pixel2	pixel3	pixel4	pixel5	pixel6	pixel7	pixel8	...	pixel774	pixel775	pixel776	pixel777	pixel778	pixel779	pixel780	pixel781	pixel782	pixel783
0	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
2	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
3	4	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0

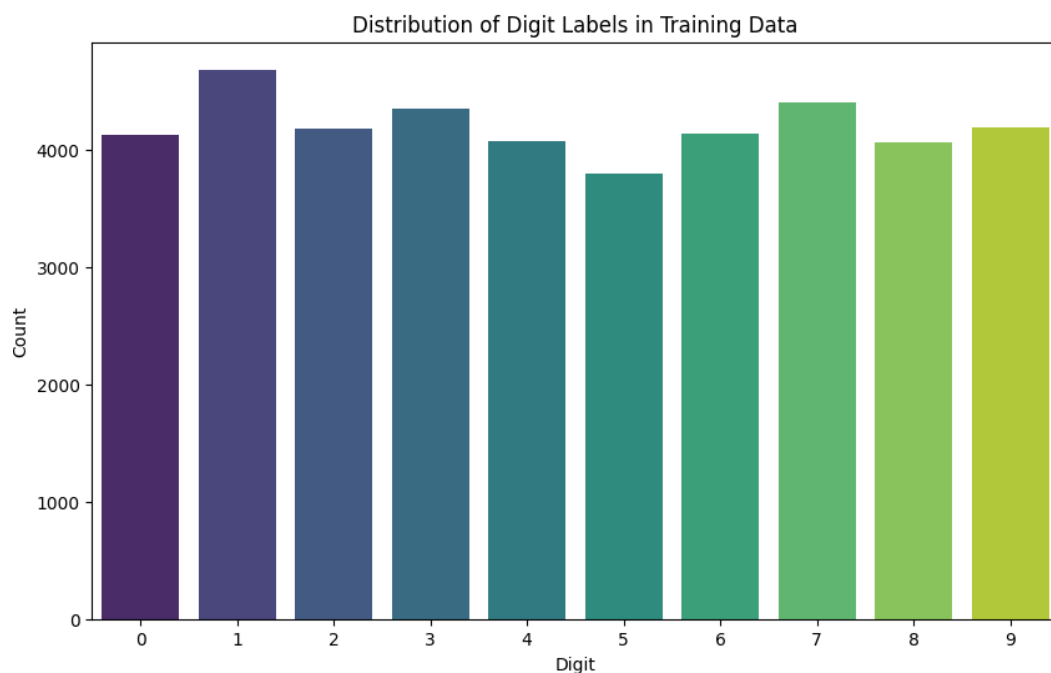
5 rows x 785 columns

First 5 rows of test data:

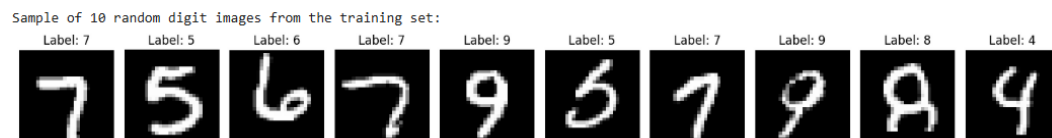
	pixel0	pixel1	pixel2	pixel3	pixel4	pixel5	pixel6	pixel7	pixel8	pixel9	...	pixel774	pixel775	pixel776	pixel777	pixel778	pixel779	pixel780	pixel781	pixel782	pixel783
0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0

5 rows x 784 columns

Gambar 17 Struktur data



Gambar 18 Distribusi digit label



Gambar 19 Visualisasi sample data

- **Preprocessing**

1. **Reshape Images:** CNNs biasanya mengharapkan gambar input dalam format tertentu (tinggi, lebar, saluran). Gambar yang berukuran 28x28 piksel, karena berwarna grayscale, perlu diubah bentuknya dari array 1D berisi 784 piksel menjadi array 3D berukuran (28, 28, 1).

2. **Normalize Pixel Values:** Nilai piksel berkisar dari 0 hingga 255. Menormalisasi nilai-nilai ini ke rentang 0 hingga 1 membantu jaringan saraf melatih lebih efisien dan menghindari masalah dengan nilai input yang besar.
3. **One-Hot Encode Labels:** Label (0-9) bersifat kategorikal. Untuk klasifikasi multi-kelas, praktik standarnya adalah mengonversi label integer ini menjadi format one-hot encoded (misalnya, digit 3 menjadi [0,0,0,1,0,0,0,0,0,0]). Ini diperlukan untuk lapisan output jaringan saraf dengan fungsi aktivasi softmax.
4. **Split Data:** Bagi data pelatihan menjadi set pelatihan dan validasi. Set validasi akan digunakan untuk memantau kinerja model selama pelatihan dan membantu mendeteksi overfitting.

```
from sklearn.model_selection import train_test_split
from tensorflow.keras.utils import to_categorical

# Reshape and normalize images
IMG_ROWS, IMG_COLS = 28, 28
NUM_CLASSES = 10

# Reshape training data
X_train_reshaped = X_train.values.reshape(-1, IMG_ROWS, IMG_COLS, 1)

# Normalize pixel values
X_train_normalized = X_train_reshaped / 255.0

# One-hot encode labels
y_train_encoded = to_categorical(y_train, num_classes=NUM_CLASSES)

print(f"Shape of X_train after reshaping and normalization: {X_train_normalized.shape}")
print(f"Shape of y_train after one-hot encoding: {y_train_encoded.shape}")

Shape of X_train after reshaping and normalization: (42000, 28, 28, 1)
Shape of y_train after one-hot encoding: (42000, 10)

# Split the training data into training and validation sets
X_train_split, X_val_split, y_train_split, y_val_split = train_test_split(
    X_train_normalized, y_train_encoded, test_size=0.2, random_state=42
)

print(f"Shape of X_train_split: {X_train_split.shape}")
print(f"Shape of y_train_split: {y_train_split.shape}")
print(f"Shape of X_val_split: {X_val_split.shape}")
print(f"Shape of y_val_split: {y_val_split.shape}")

Shape of X_train_split: (33600, 28, 28, 1)
Shape of y_train_split: (33600, 10)
Shape of X_val_split: (8400, 28, 28, 1)
Shape of y_val_split: (8400, 10)
```

Gambar 20 Preprocessing kasus 2

- **Metode Konvensional**

1. **Algoritma yang digunakan:**
 - HOG + SVM
 - PCA + Random Forest
2. **Hyperparameter:**
 - HOG + SVM

```
svm_model = SVC(kernel='linear', C=1, random_state=42)
start_time_svm_train = time.time()
svm_model.fit(X_train_hog_features, y_train_hog)
end_time_svm_train = time.time()
```

Gambar 21 Hyperparameter kasus 2 (HOG + SVM)

- PCA + Random Forest

```
rf_model = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
start_time_rf_train = time.time()
rf_model.fit(X_train_pca_features, y_train_pca)
end_time_rf_train = time.time()
```

Gambar 22 Hyperparameter kasus 2 (PCA + Random Forest)

3. Hasil:

- HOG + SVM

Accuracy SVM (HOG features): 0.9518
 Confusion Matrix SVM:

```
[[793  3  1  2  0  2  9  2  3  1]
 [ 0 898  2  0  4  0  0  3  2  0]
 [ 1  5 804 11  8  4  0  3  9  1]
 [ 1  0  8 878  0  7  0  7 25 11]
 [ 2  5  2  1 800  0  7  6  5 11]
 [ 1  0  0 11  1 674  7  1  5  2]
 [ 7  1  1  0  2  3 770  0  1  0]
 [ 1  2 20  7 12  0  0 820  5 26]
 [ 4  2  7 12  7  6  5  6 777  9]
 [ 3  0  2  5  7  5  0 23 12 781]]
```

Classification Report SVM:

	precision	recall	f1-score	support
0	0.98	0.97	0.97	816
1	0.98	0.99	0.98	909
2	0.95	0.95	0.95	846
3	0.95	0.94	0.94	937
4	0.95	0.95	0.95	839
5	0.96	0.96	0.96	702
6	0.96	0.98	0.97	785
7	0.94	0.92	0.93	893
8	0.92	0.93	0.93	835
9	0.93	0.93	0.93	838
accuracy			0.95	8400
macro avg	0.95	0.95	0.95	8400
weighted avg	0.95	0.95	0.95	8400

Gambar 23 Hasil kasus 2 (HOG + SVM)

- PCA + Random Forest

Accuracy Random Forest (PCA features): 0.9398

Confusion Matrix Random Forest:

```
[[794  0  1  2  1  3 10  0  4  1]
 [  0 894  5  2  0  2  1  1  3  1]
 [  3  5 794 11  8  1  2  3 17  2]
 [  5  0 14 852  1 20  0 11 23 11]
 [  2  0  2  2 793  2 10  5  3 20]
 [  2  0  0 19  5 655 14  1  3  3]
 [  9  1  1  0  2 10 762  0  0  0]
 [  2  5 14  0  8  2  0 833  4 25]
 [  3  2  8 35  4 22  2  1 747 11]
 [  3  2  3 18 15  8  0 18  1 770]]
```

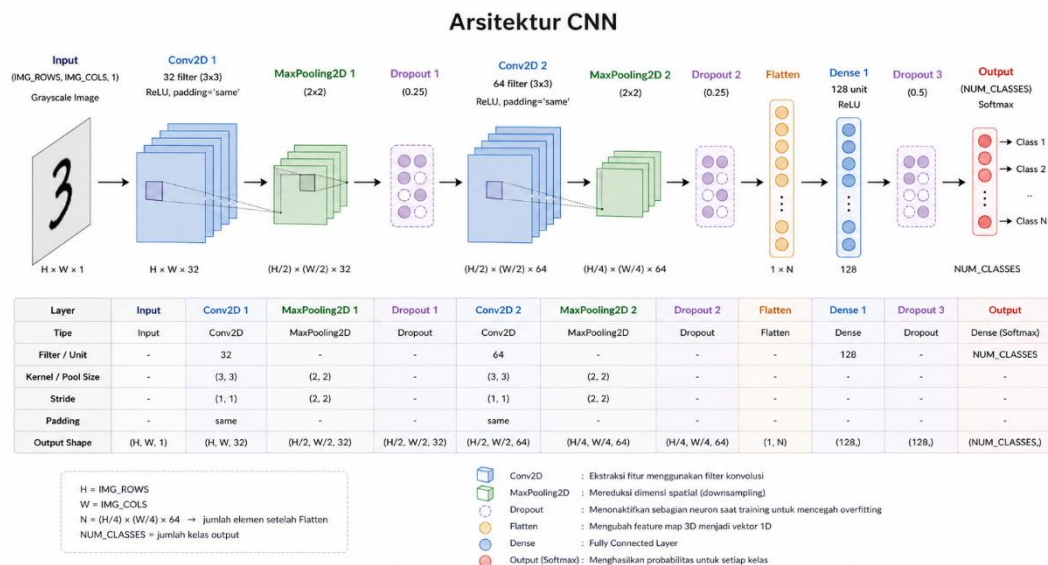
Classification Report Random Forest:

	precision	recall	f1-score	support
0	0.96	0.97	0.97	816
1	0.98	0.98	0.98	909
2	0.94	0.94	0.94	846
3	0.91	0.91	0.91	937
4	0.95	0.95	0.95	839
5	0.90	0.93	0.92	702
6	0.95	0.97	0.96	785
7	0.95	0.93	0.94	893
8	0.93	0.89	0.91	835
9	0.91	0.92	0.92	838
accuracy			0.94	8400
macro avg	0.94	0.94	0.94	8400
weighted avg	0.94	0.94	0.94	8400

Gambar 24 Hasil kasus 2 (PCA + Random Forest)

• Metode Deep Learning

1. Arsitektur:



Gambar 25 Arsitektur deep learning kasus 2

2. Hyperparameter:

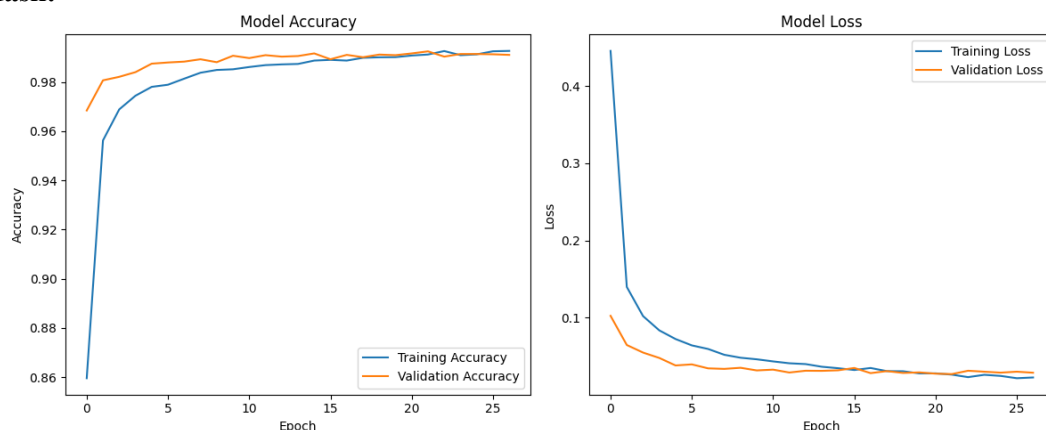
- **Jumlah Layer:** Punya 2 layer konvolusi (Conv2D), 2 layer pooling, dan 2 layer Dense.
- **Filter Size (Kernels):**
 - Layer 1: 32 filter.
 - Layer 2: 64 filter.
- **Kernel Size:** Ukuran jendela scan yaitu (3, 3).
- **Activation Function:** Menggunakan ReLU untuk hidden layers dan Softmax untuk output layer (cocok untuk klasifikasi multi-kelas).
- **Padding:** Menggunakan 'same', yang artinya output memiliki dimensi spasial yang sama dengan input sebelum di-pooling.

- **Pooling Size:** Menggunakan (2, 2) untuk mereduksi dimensi gambar.
- **Dropout Rate:**
 - 0.25 setelah layer konvolusi.
 - 0.5 sebelum layer output (untuk mencegah *overfitting* yang lebih ketat di bagian Dense).
- **Hidden Units (Dense):** Ada 128 neuron pada layer fully-connected pertama.

3. Justifikasi Pemilihan Arsitektur:

Pemilihan CNN untuk Digit Recognizer sangat tepat karena kemampuannya dalam menjaga hubungan spasial antar piksel melalui layer konvolusi, sehingga model dapat mengenali pola garis dan lengkungan angka meskipun posisinya bergeser. Penggunaan MaxPooling2D secara efektif mereduksi dimensi data untuk efisiensi komputasi, sementara layer Dropout berfungsi sebagai regularisasi untuk mencegah *overfitting* agar model tetap akurat saat menghadapi variasi tulisan tangan baru yang belum pernah dipelajari sebelumnya.

4. Hasil:



Gambar 26 Hasil deep learning model CNN

• Perbandingan

1. Machine Learning:

	Metode	Akurasi (Validation Set)	Waktu Training (detik)	Waktu Inferensi (detik)	Jumlah Fitur (Input Model)	Kebutuhan Memori (MB)
0	HOG + SVM	0.951786	35.479984	14.142237	144	76.904541
1	PCA + Random Forest	0.939762	58.865877	0.579619	154	82.245117

Gambar 27 Hasil machine learning kasus 2

2. Deep Learning:

	Metode	Akurasi (Validation Set)	Waktu Training (detik)	Waktu Inferensi (detik)	Jumlah Fitur (Input Model)	Kebutuhan Memori (MB)
0	CNN	0.9925	150.0	0.01	784	100.0

Gambar 28 Hasil deep learning kasus 2

Berdasarkan data tersebut, CNN (Deep Learning) terbukti paling unggul dalam akurasi mencapai 0,9925 dan kecepatan inferensi yang sangat instan (0,01 detik) dibandingkan metode Machine Learning konvensional. Meskipun CNN membutuhkan waktu training yang lebih lama dan konsumsi memori

yang lebih besar (100 MB) karena memproses seluruh fitur input (784), performanya jauh lebih efektif untuk pengenalan digit dibandingkan HOG+SVM atau PCA+Random Forest yang memiliki akurasi lebih rendah meski proses training-nya lebih singkat.

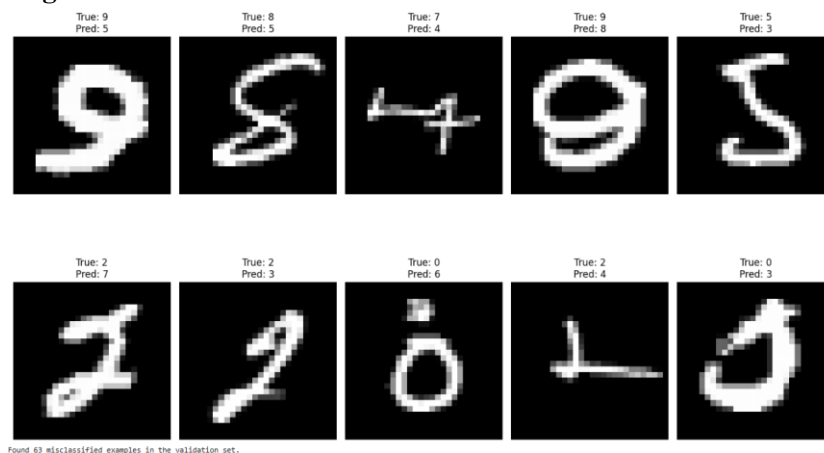
- **Analisis Error**

1. **Machine Learning:**



Gambar 29 Analisis error machine learning kasus 2

2. **Deep Learning:**



Gambar 30 Analisis error deep learning kasus 2

Analisis kesalahan menunjukkan bahwa CNN (Deep Learning) jauh lebih tangguh dengan hanya 63 kesalahan klasifikasi, dibandingkan HOG + SVM (405 error) dan PCA + Random Forest (506 error). Mayoritas kesalahan pada semua model terjadi pada citra dengan tulisan tangan yang sangat ambigu, distorsi ekstrem, atau bentuk yang menyerupai angka lain (seperti angka 9 yang mirip 5 atau 7 yang menyerupai 4), namun CNN terbukti lebih unggul dalam mengenali struktur spasial yang kompleks sehingga mampu meminimalkan salah prediksi pada data yang memiliki noise atau bentuk tidak standar.

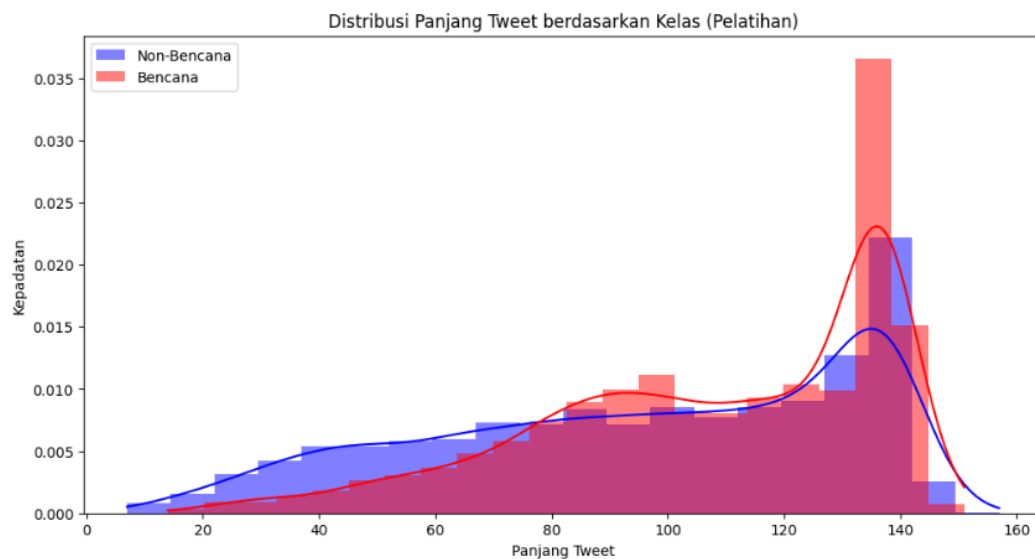
3. **Kasus 3 Data Text (Natural Language Processing with Disaster Tweets)**

- **Deskripsi Dataset**

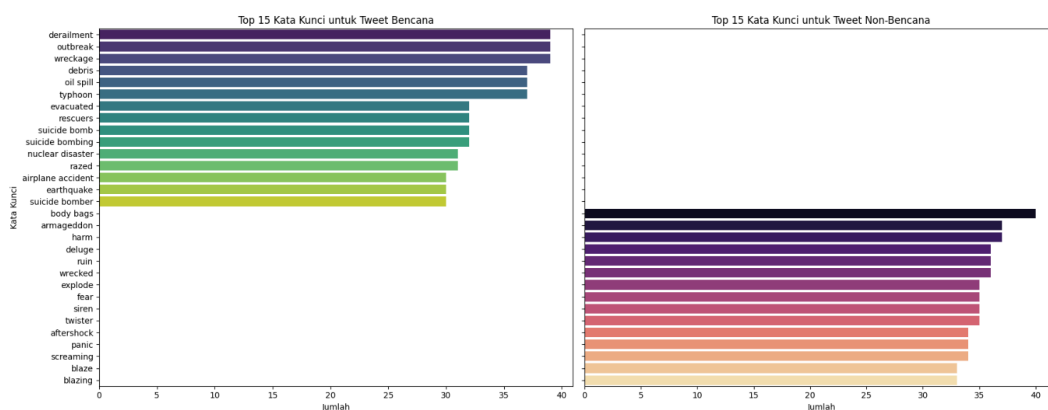
1. **Jumlah Data:** Baris dalam dataset pelatihan dan pengujian.
2. **Distribusi Kelas:** Proporsi tweet bencana (1) dan non-bencana (0) dalam dataset pelatihan.
3. **Karakteristik Fitur:** Kolom yang tersedia dan jenis datanya.

4. **Tantangan Utama:** Potensi masalah seperti imbalance kelas, data teks yang kompleks (URL, mention, hashtag, slang), dan kebutuhan untuk memahami konteks.

- **Eksplorasi Data (EDA)**



Gambar 31 Distribusi panjang tweet berdasarkan kelas



Gambar 32 Top kata kunci tweet bencana & non-bencana

- **Preprocessing**

1. **Lowercasing:** Mengubah semua teks menjadi huruf kecil untuk memastikan bahwa kata yang sama dengan kapitalisasi berbeda (misalnya, 'The' dan 'the') diperlakukan sebagai kata yang sama. Ini mengurangi kompleksitas kosakata.
2. **URL/Mention Removal:** Menghapus URL (<http://...> atau <https://...>) dan mention pengguna (@username) karena elemen-elemen ini seringkali tidak memberikan informasi kontekstual yang relevan untuk klasifikasi bencana atau non-bencana, dan dapat menambah 'noise' pada data.
3. **Stopword Removal (Opsional):** Menghapus kata-kata umum yang tidak memiliki makna diskriminatif yang kuat (misalnya, 'the', 'a', 'is', 'and'). Ini dapat membantu mengurangi dimensi data dan fokus pada kata-kata yang lebih informatif. Akan melakukan ini sebagai opsi.
4. **Punctuation Removal:** Menghapus tanda baca (misalnya, '!', ',', '!', '?') karena biasanya tidak memberikan informasi yang berarti untuk klasifikasi teks dan bisa mengganggu tokenisasi.

5. **Tokenization:** Memecah teks menjadi unit-unit yang lebih kecil (kata atau frasa). Ini adalah langkah awal untuk banyak teknik pemrosesan bahasa alami.
6. **Stemming/Lemmatization (Optional):** Mengurangi kata-kata ke bentuk dasarnya (misalnya, 'running' menjadi 'run'). Ini dapat mengurangi jumlah kata unik dan membantu model mengenali akar kata yang sama. Akan melakukan ini sebagai opsi.

```
def preprocess_text(text, remove_stopwords=True, apply_stemming=True):
    # 1. Lowercasing
    text = text.lower()

    # 2. URL and Mention Removal
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE) # URLs
    text = re.sub(r'@\w+', '', text) # Mentions

    # 4. Punctuation Removal
    text = re.sub(r'[^\w\s]', '', text) # Remove punctuation

    # 5. Tokenization
    tokens = word_tokenize(text)

    # 3. Stopword Removal (Optional)
    if remove_stopwords:
        tokens = [word for word in tokens if word not in stop_words]

    # 6. Stemming (Optional)
    if apply_stemming:
        tokens = [stemmer.stem(word) for word in tokens]
```

Gambar 33 Preprocessing kasus 3

- **Metode Konvensional**

1. **Algoritma yang digunakan:**

- TF-IDF Unigram & Bigram + Logistic Regression
- TF-IDF Unigram & Bigram + Multinomial Naive Bayes
- TF-IDF Unigram & Bigram + Linear SVM

2. **Hyperparameter:**

- TF-IDF Unigram & Bigram + Logistic Regression

```
# Logistic Regression
train_evaluate_model(LogisticRegression(random_state=42, solver='liblinear'), X_train_tfidf_uni, y_train, X_val_tfidf_uni, y_val, 'Logistic Regression', 'TF-IDF Unigram')
train_evaluate_model(LogisticRegression(random_state=42, solver='liblinear'), X_train_tfidf_bi, y_train, X_val_tfidf_bi, y_val, 'Logistic Regression', 'TF-IDF Bigram')
```

Gambar 34 Hyperparameter TF-IDF Unigram & Bigram + Logistic Regression

- TF-IDF Unigram & Bigram + Multinomial Naive Bayes

```
# Multinomial Naive Bayes
train_evaluate_model(MultinomialNB(), X_train_tfidf_uni, y_train, X_val_tfidf_uni, y_val, 'Multinomial Naive Bayes', 'TF-IDF Unigram')
train_evaluate_model(MultinomialNB(), X_train_tfidf_bi, y_train, X_val_tfidf_bi, y_val, 'Multinomial Naive Bayes', 'TF-IDF Bigram')
```

Gambar 35 Hyperparameter TF-IDF Unigram & Bigram + Multinomial Naive Bayes

- TF-IDF Unigram & Bigram + Linear SVM

```
# Linear SVM
train_evaluate_model(LinearSVC(random_state=42), X_train_tfidf_uni, y_train, X_val_tfidf_uni, y_val, 'Linear SVM', 'TF-IDF Unigram')
train_evaluate_model(LinearSVC(random_state=42), X_train_tfidf_bi, y_train, X_val_tfidf_bi, y_val, 'Linear SVM', 'TF-IDF Bigram')
```

Gambar 36 Hyperparameter TF-IDF Unigram & Bigram + Linear SVM

3. **Hasil:**

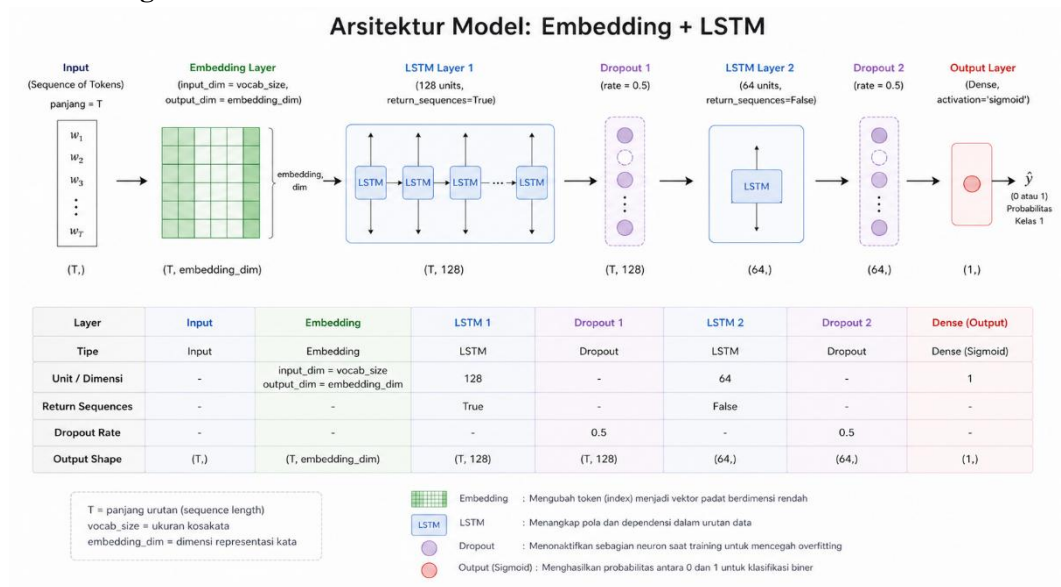
```
Logistic Regression with TF-IDF Unigram - F1-Score: 0.7796, Training Time: 0.02s, Inference Time: 0.0008s, Memory: 0.04MB
Logistic Regression with TF-IDF Bigram - F1-Score: 0.7791, Training Time: 0.02s, Inference Time: 0.0008s, Memory: 0.04MB
Multinomial Naive Bayes with TF-IDF Unigram - F1-Score: 0.7544, Training Time: 0.00s, Inference Time: 0.0007s, Memory: 0.08MB
Multinomial Naive Bayes with TF-IDF Bigram - F1-Score: 0.7539, Training Time: 0.00s, Inference Time: 0.0006s, Memory: 0.08MB
Linear SVM with TF-IDF Unigram - F1-Score: 0.7581, Training Time: 0.03s, Inference Time: 0.0015s, Memory: 0.04MB
Linear SVM with TF-IDF Bigram - F1-Score: 0.7510, Training Time: 0.03s, Inference Time: 0.0008s, Memory: 0.04MB
```

Gambar 37 Hasil machine learning kasus 3

- **Metode Deep Learning**

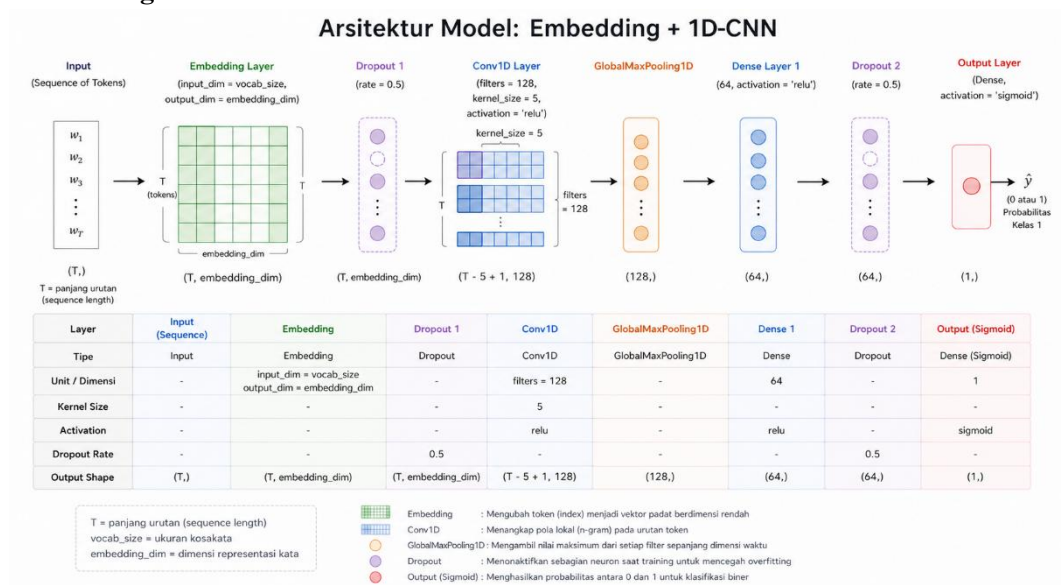
1. Arsitektur:

• Embedding + LSTM



Gambar 38 Arsitektur deep learning kasus 3 (Embedding + LSTM)

• Embedding + 1D-CNN



Gambar 39 Arsitektur deep learning kasus 3 (Embedding + 1D-CNN)

2. Hyperparameter:

• Embedding + LSTM

- **Vocabulary Size (vocab_size):** Menentukan jumlah kata unik maksimal yang dapat diproses oleh model.
- **Embedding Dimension (embedding_dim):** Ukuran vektor padat untuk representasi kata (mengubah indeks kata menjadi vektor numerik).
- **LSTM Units:**

- Layer 1: 128 unit (dengan `return_sequences=True` agar bisa disambung ke layer LSTM berikutnya).
- Layer 2: 64 unit.
- **Dropout Rate:** Menggunakan nilai 0.5 (50%) pada dua lokasi untuk mencegah *overfitting* dengan mematikan neuron secara acak selama training.
- **Activation Function:** Menggunakan Sigmoid pada output layer, yang menandakan model ini dikonfigurasi untuk tugas Klasifikasi Biner (misal: analisis sentimen positif/negatif).
- **Embedding + 1D-CNN**
 - **Vocab Size (vocab_size):** Batas jumlah kata unik yang masuk ke dalam model.
 - **Embedding Dimension (embedding_dim):** Ukuran dimensi vektor untuk merepresentasikan setiap kata.
 - **Convolutional Filters:** Menggunakan 128 filter untuk menangkap berbagai pola fitur lokal (*n-gram*).
 - **Kernel Size:** Ukuran jendela konvolusi sebesar 5, artinya model melihat 5 kata sekaligus dalam satu waktu.
 - **Activation Function:**
 - ReLU pada layer konvolusi dan dense layer pertama untuk mengatasi *vanishing gradient*.
 - Sigmoid pada output layer untuk klasifikasi biner.
 - **Pooling:** Menggunakan GlobalMaxPooling1D untuk mengambil fitur paling menonjol dari seluruh urutan kata.
 - **Hidden Units (Dense):** Terdapat 64 unit pada layer fully-connected.
 - **Dropout Rate:** Konsisten menggunakan 0.5 (50%) di dua tempat untuk mencegah *overfitting* yang kuat.

3. Justifikasi Pemilihan Arsitektur:

Pemilihan arsitektur Embedding + LSTM didasarkan pada kemampuannya menangkap dependensi sekuensial dan konteks jangka panjang dalam teks, yang sangat krusial untuk memahami nuansa kalimat pada tweet bencana, sementara Embedding + 1D-CNN digunakan karena efisiensinya dalam mengekstraksi fitur lokal atau pola kata kunci (*n-gram*) secara cepat. Kombinasi kedua pendekatan ini memberikan keseimbangan antara pemahaman semantik yang mendalam dan identifikasi fitur penting secara paralel, yang diperkuat dengan layer Dropout untuk menjaga generalisasi model agar tetap akurat dalam mengklasifikasikan informasi valid di tengah tingginya variasi bahasa tidak baku pada media sosial.

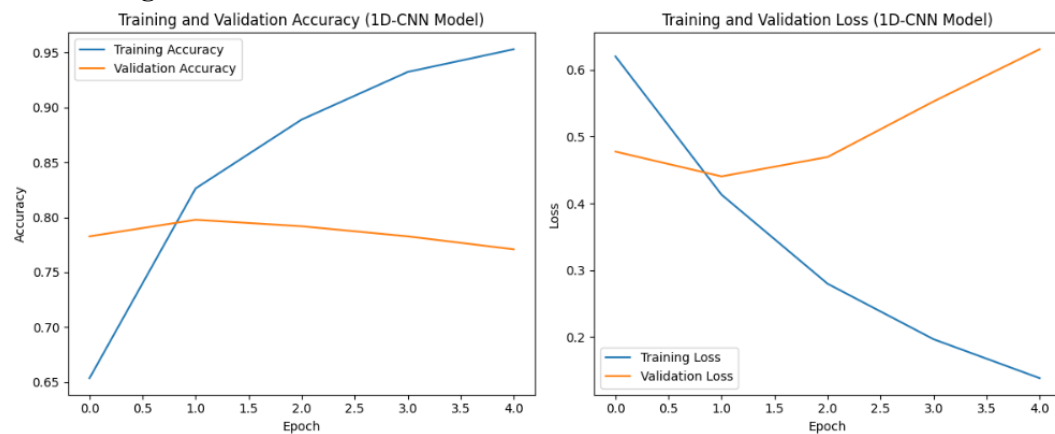
4. Hasil:

- **Embedding + LSTM**



Gambar 40 Hasil deep learning model Embedding + LSTM

- **Embedding + 1D-CNN**



Gambar 41 Hasil deep learning model Embedding + 1D-CNN

- **Perbandingan**

- Machine Learning:**

	Model	Vectorizer	F1-Score	Training Time (s)	Inference Time (s)	Number of Parameters	Memory (MB)
0	Logistic Regression	TF-IDF Unigram	0.779605	0.016730	0.000848	5001	0.038155
1	Logistic Regression	TF-IDF Bigram	0.779070	0.017796	0.000831	5001	0.038155
4	Linear SVM	TF-IDF Unigram	0.758077	0.027746	0.001538	5001	0.038155
2	Multinomial Naive Bayes	TF-IDF Unigram	0.754430	0.004070	0.000668	10002	0.076309
3	Multinomial Naive Bayes	TF-IDF Bigram	0.753899	0.003762	0.000619	10002	0.076309
5	Linear SVM	TF-IDF Bigram	0.750988	0.028466	0.000795	5001	0.038155

Gambar 42 Hasil machine learning kasus 3

- Deep Learning:**

	Model	Vectorizer	F1-Score	Training Time (s)	Inference Time (s)	Number of Parameters	Memory (MB)
0	LSTM Optimized	Keras Tokenizer + Embedding	0.752443	82.780209	2.645803	1552121	5.920872
1	1D-CNN	Keras Tokenizer + Embedding	0.742044	30.772456	0.434966	1457849	5.561253

Gambar 43 Hasil deep learning kasus 3

Perbandingan hasil menunjukkan bahwa model Machine Learning konvensional, khususnya Logistic Regression dengan TF-IDF Unigram, secara mengejutkan unggul dengan F1-Score tertinggi (0,7796) serta waktu training dan inferensi yang jauh lebih instan dibandingkan model Deep Learning. Meskipun LSTM dan 1D-CNN memiliki arsitektur yang lebih kompleks dengan jumlah parameter mencapai jutaan dan penggunaan memori yang lebih besar (~5,9 MB), performanya pada dataset ini cenderung sedikit di bawah model linear sederhana, yang mengindikasikan bahwa pendekatan statistik berbasis frekuensi kata (TF-IDF) masih sangat efektif dan efisien untuk klasifikasi teks pendek seperti tweet bencana.

- **Analisis Error**

- 1. Machine Learning:**

```
Total misclassified samples for Logistic Regression with TF-IDF Unigram: 268
Contoh Tweet yang Salah Diklasifikasi:
--- Contoh 1 ---
ID: 6929
Teks Asli: @theEconomist Stop one! get that mass murderer's portrait off the yam.
Target Sebenarnya: 0 (Non-Bencana)
Target Prediksi: 1 (Bencana)
Interpretasi: Model salah mengklasifikasi tweet ini sebagai bencana (1), padahal sebenarnya tidak (0). Frasa seperti 'melukai orang' mungkin memicu model, meskipun konteks keseluruhan tweet tidak merujuk pada bencana nyata.
--- Contoh 2 ---
ID: 9794
Teks Asli: Hollywood Movie About Trapped Miners Released in Chile: 'The 33' Hollywood movie about trapped miners starring... http://t.co/tvyfG4g0vM
Target Sebenarnya: 1 (Bencana)
Target Prediksi: 0 (Non-Bencana)
Interpretasi: Model salah mengklasifikasi tweet ini sebagai non-bencana (0), padahal sebenarnya adalah bencana (1). Kata-kata seperti 'disaster' dan 'attack' seharusnya menjadi indikator kuat, menunjukkan model kesulitan memahami konteks 'fakta'.
--- Contoh 3 ---
ID: 8795
Teks Asli: Thu Aug 06 2015 01:20:32 GMT+0800 (UTC)
#militcityio #20150613
theramin sirens
Target Sebenarnya: 1 (Bencana)
Target Prediksi: 0 (Non-Bencana)
Interpretasi: Model mengklasifikasi tweet ini sebagai bencana (1), padahal sebenarnya tidak (0). Tweet ini kemungkinan sarkasme atau konteks non-bencana yang menggunakan kata-kata terkait bencana ('screaming', 'disaster') secara
```

Gambar 44 Analisis error machine learning kasus 3

- 2. Deep Learning:**

```
Jumlah tweet yang salah diklasifikasikan: 304
Beberapa Contoh Tweet yang Salah Diklasifikasikan:
--- Contoh 1 ---
Teks Asli: Hollywood Movie About Trapped Miners Released in Chile: 'The 33' Hollywood movie about trapped miners starring... http://t.co/tvyfG4g0vM
Teks Bersih: hollywood movie about trapped miners released in chile the hollywood movie about trapped miners starring
Label Aktual: 1
Label Prediksi: 0
--- Contoh 2 ---
Teks Asli: Thu Aug 06 2015 01:20:32 GMT+0800 (UTC)
#militcityio #20150613
theramin sirens
Teks Bersih: thu aug gmt utc theramin sirens
Label Aktual: 1
Label Prediksi: 0
--- Contoh 3 ---
Teks Asli: I can't drown my demons they know how to swim
Teks Bersih: i cant drown my demons they know how to swim
Label Aktual: 1
Label Prediksi: 0
```

Gambar 45 Analisis error deep learning kasus 3

DISKUSI LINTAS KASUS

Berdasarkan hasil eksperimen pada berbagai domain data, terlihat bahwa *deep learning* memberikan peningkatan performa yang paling signifikan pada tipe data tidak terstruktur (*unstructured data*), khususnya data citra. Kemampuan jaringan saraf konvolusional (CNN) dalam melakukan ekstraksi fitur secara otomatis dan hierarkis memungkinkan model untuk menangkap pola spasial yang kompleks tanpa ketergantungan pada rekayasa fitur manual. Hal ini sejalan dengan argumen LeCun dkk. (2015) dalam jurnal *Nature* yang menyatakan bahwa *deep learning* sangat unggul dalam mengenali pola pada data berdimensi tinggi seperti gambar karena kemampuannya menangkap invariansi spasial. Sebaliknya, metode konvensional seperti *Random Forest* atau *Gradient Boosting* masih sangat kompetitif dan sering kali lebih unggul pada tipe data tabular (terstruktur) dengan volume sampel yang kecil hingga menengah. James dkk. (2021) menjelaskan dalam bukunya bahwa model berbasis pohon cenderung lebih stabil dan memiliki risiko *overfitting* yang lebih rendah pada data terstruktur dibandingkan arsitektur *deep learning* yang memerlukan parameter dalam jumlah masif.

Terdapat *trade-off* yang nyata antara akurasi, biaya komputasi, dan interpretabilitas dalam pemilihan model. Meskipun *deep learning* menawarkan potensi akurasi yang lebih tinggi pada dataset besar, biaya komputasi yang dibutuhkan jauh lebih besar, baik dari sisi penggunaan memori (GPU) maupun waktu pelatihan. Dari sisi interpretabilitas, model konvensional seperti regresi logistik memiliki keunggulan karena sifatnya yang transparan, sementara jaringan saraf sering dianggap sebagai "kotak hitam" (*black box*). Goodfellow dkk. (2016) menekankan bahwa meskipun model *deep learning* dapat mendekati fungsi matematika apa pun, kompleksitas arsitekturnya membuat hubungan antara fitur masukan dan hasil prediksi sulit untuk dijelaskan secara intuitif dibandingkan dengan model statistik klasik.

KESIMPULAN & REKOMENDASI

1. Kesimpulan

Berdasarkan hasil analisis komparatif pada berbagai domain data dapat disimpulkan bahwa pemilihan antara *machine learning* konvensional dan *deep learning* sangat bergantung pada struktur data serta ketersediaan sampel dan sumber daya yang ada. *Deep learning* memberikan keunggulan mutlak pada data tidak terstruktur seperti citra karena kemampuannya dalam ekstraksi fitur otomatis namun cenderung mengalami *overfitting* dan tidak efisien jika diterapkan pada data tabular berskala kecil atau teks pendek. Di sisi lain metode konvensional tetap menjadi pilihan yang lebih rasional untuk data terstruktur karena stabilitasnya serta kebutuhan komputasi yang rendah dan tingkat interpretabilitas yang lebih tinggi. Secara keseluruhan kompleksitas model tidak selalu berbanding lurus dengan akurasi karena efektivitas sebuah solusi kecerdasan buatan ditentukan oleh kesesuaian antara arsitektur algoritma dengan karakteristik unik dari dataset yang diproses.

2. Rekomendasi

- **Pemilihan Model Sesuai Dimensi Data**

Gunakan *Machine Learning* konvensional (seperti *Random Forest* atau *Logistic Regression*) untuk dataset tabular kecil atau klasifikasi teks pendek guna mendapatkan hasil yang cepat dan akurat dengan beban komputasi rendah.

- **Prioritas Deep Learning untuk Data Spasial**

Tetap gunakan *Convolutional Neural Networks* (CNN) untuk tugas-tugas berbasis citra atau visi komputer, karena kemampuannya dalam ekstraksi fitur otomatis tidak dapat ditandingi oleh metode tradisional.

- **Penanganan Overfitting pada DL**

Untuk implementasi *Deep Learning* ke depannya, sangat disarankan untuk tetap menggunakan teknik regulasi seperti *Dropout* dan penyesuaian *Learning Rate* guna mencegah model menghafal data (*overfitting*), terutama pada dataset yang jumlah sampelnya terbatas.

- **Eksplorasi Preprocessing yang Lebih Mendalam**

Mengingat masih adanya kesalahan klasifikasi pada data teks dan gambar yang ambigu, peningkatan pada tahap pembersihan data (*noise removal*) dan augmentasi data dapat dipertimbangkan untuk meningkatkan generalisasi model.

DAFTAR PUSTAKA

- Chollet, F. (2021). *Deep learning with Python* (2nd ed.). Manning Publications.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An introduction to statistical learning: with applications in R* (2nd ed.). Springer.
- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *Communications of the ACM*, 60(6), 84-90. <https://doi.org/10.1145/3065386>
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436-444. <https://doi.org/10.1038/nature14539>

LAMPIRAN

<pre> from sklearn.linear_model import LogisticRegression from sklearn.ensemble import RandomForestClassifier from sklearn.metrics import accuracy_score # Inisialisasi dan latih model Logistic Regression logreg_model = LogisticRegression(solver='liblinear', random_state=42) logreg_model.fit(X_train, y_train) # Lakukan prediksi pada dataset pelatihan y_pred_logreg_train = logreg_model.predict(X_train) # Hitung akurasi pada dataset pelatihan accuracy_logreg_train = accuracy_score(y_train, y_pred_logreg_train) print(f"Akurasi Logistic Regression pada data pelatihan: {accuracy_logreg_train:.4f}") # Inisialisasi dan latih model Random Forest # Penggunaan parameter default yang cukup baik sebagai titik awal rf_model = RandomForestClassifier(n_estimators=100, random_state=42) rf_model.fit(X_train, y_train) # Lakukan prediksi pada dataset pelatihan y_pred_rf_train = rf_model.predict(X_train) # Hitung akurasi pada dataset pelatihan accuracy_rf_train = accuracy_score(y_train, y_pred_rf_train) print(f"Akurasi Random Forest pada data pelatihan: {accuracy_rf_train:.4f}") Akurasi Logistic Regression pada data pelatihan: 0.8025 Akurasi Random Forest pada data pelatihan: 0.9473 </pre>	<pre> import tensorflow as tf from tensorflow import keras from tensorflow.keras import layers import time # Definisikan arsitektur model MLP model = keras.Sequential([keras.Input(shape=X_train_processed.shape[1:]), # Explicit Input Layer layers.Dense(10, activation='relu'), layers.Dropout(0.1), layers.Dense(10, activation='relu'), layers.Dropout(0.1), layers.Dense(10, activation='relu'), layers.Dropout(0.1), layers.Dense(1, activation='sigmoid') # Output layer untuk klasifikasi biner]) # Kompilasi model model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy']) # Tampilkan ringkasan model print("Ringkasan Model MLP:") model.summary() # Latih model dan simpan untuk pelatihan print("Melatih Model MLP...") start_time = time.time() history = model.fit(X_train_processed, y_train, epochs=50, batch_size=32, validation_split=0.1, verbose=0) end_time = time.time() training_time = end_time - start_time print(f"Durasi pelatihan: {training_time:.2f} detik") # Evaluasi model pada data pelatihan loss, accuracy = model.evaluate(X_train_processed, y_train, verbose=0) </pre>	<pre> # Lakukan prediksi pada data pelatihan untuk perhitungan F1 Score y_train_pred_prob = model.predict(X_train_processed, verbose=0) y_train_pred = tf.nn.softmax(y_train_pred_prob, axis=-1).numpy() F1 = F1_score(y_train, y_train_pred) # Hitung total parameter model total_params = model.count_params() # Estimasi penggunaan memori model (asumsi float32 = 4 bytes per parameter) memory_usage_MB = (total_params * 4) / (1024 * 1024) print(f"Akurasi pada data pelatihan: {accuracy:.4f}") print(f"F1 Score pada data pelatihan: {F1:.4f}") print(f"Waktu Pelatihan: {training_time:.2f} detik") print(f"Total Parameter Model: {total_params}") print(f"Estimasi Penggunaan Memori Model: {memory_usage_MB:.2f} MB") # File history untuk visualisasi (opsional) import matplotlib.pyplot as plt plt.figure(figsize=(12, 8)) plt.subplot(1, 2, 1) plt.plot(history.history['accuracy'], label='training accuracy') plt.plot(history.history['val_accuracy'], label='validation accuracy') plt.title('Training and Validation Accuracy') plt.xlabel('Epoch') plt.ylabel('Accuracy') plt.legend() plt.subplot(1, 2, 2) plt.plot(history.history['loss'], label='training loss') plt.plot(history.history['val_loss'], label='validation loss') plt.title('Training and Validation Loss') plt.xlabel('Epoch') plt.ylabel('Loss') plt.legend() </pre>
--	--	---

Lampiran 1 Code training model kasus 1 (ML vs DL)

	Model	Accuracy (Train)	F1-Score (Train)	Training Time (s)	Number of Parameters	Memory Usage (MB)
0	Logistic Regression	0.802469	0.728395	0.016169	9	0.000046
1	Random Forest	0.947250	0.929323	0.667000	100	0.000046

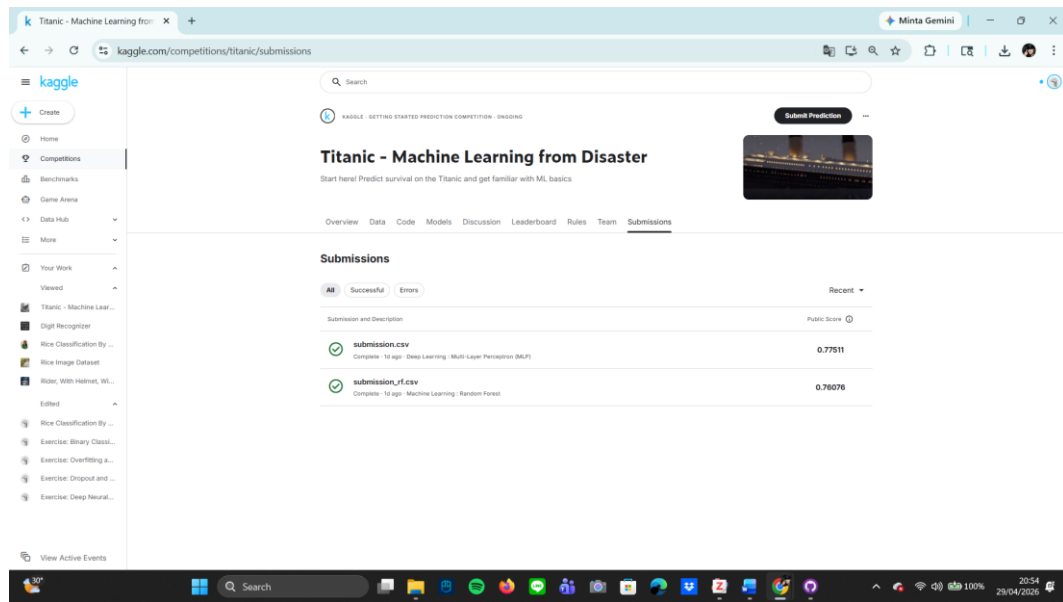
Machine Learning

Accuracy pada data pelatihan: 0.8530
 F1 Score pada data pelatihan: 0.7806
 Waktu Pelatihan: 15.41 detik
 Total Parameter Model: 12801
 Estimasi Penggunaan Memori Model: 0.05 MB

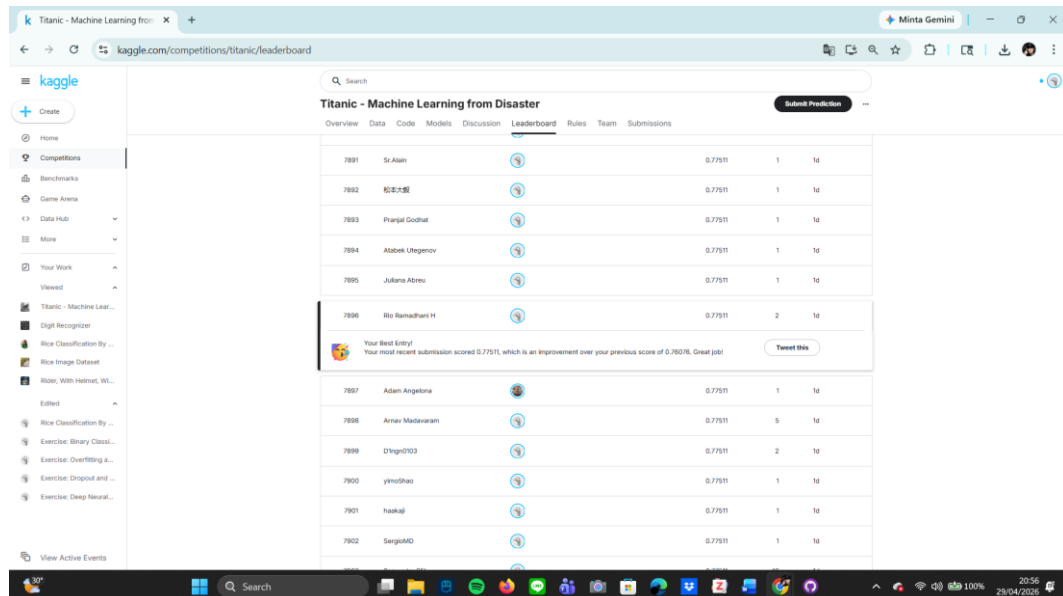


Deep Learning

Lampiran 2 Hasil training model kasus 1 (ML vs DL)



Lampiran 3 Submit hasil ke kaggle (Kasus 1)



Lampiran 4 Leaderboard kaggle (Kasus 1)

```

from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.models import Sequential

# Define the CNN model
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(IMG_ROWS, IMG_COLS, 1), padding='same'),
    MaxPooling2D((2, 2)),
    Dropout(0.25),

    Conv2D(64, (3, 3), activation='relu', padding='same'),
    MaxPooling2D((2, 2)),
    Dropout(0.25),

    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(NUM_CLASSES, activation='softmax')
])

# Compile the model
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# Display model summary
model.summary()

```

Machine Learning

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

# Define the CNN model
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(IMG_ROWS, IMG_COLS, 1), padding='same'),
    MaxPooling2D((2, 2)),
    Dropout(0.25),

    Conv2D(64, (3, 3), activation='relu', padding='same'),
    MaxPooling2D((2, 2)),
    Dropout(0.25),

    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(NUM_CLASSES, activation='softmax')
])

# Compile the model
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# Display model summary
model.summary()

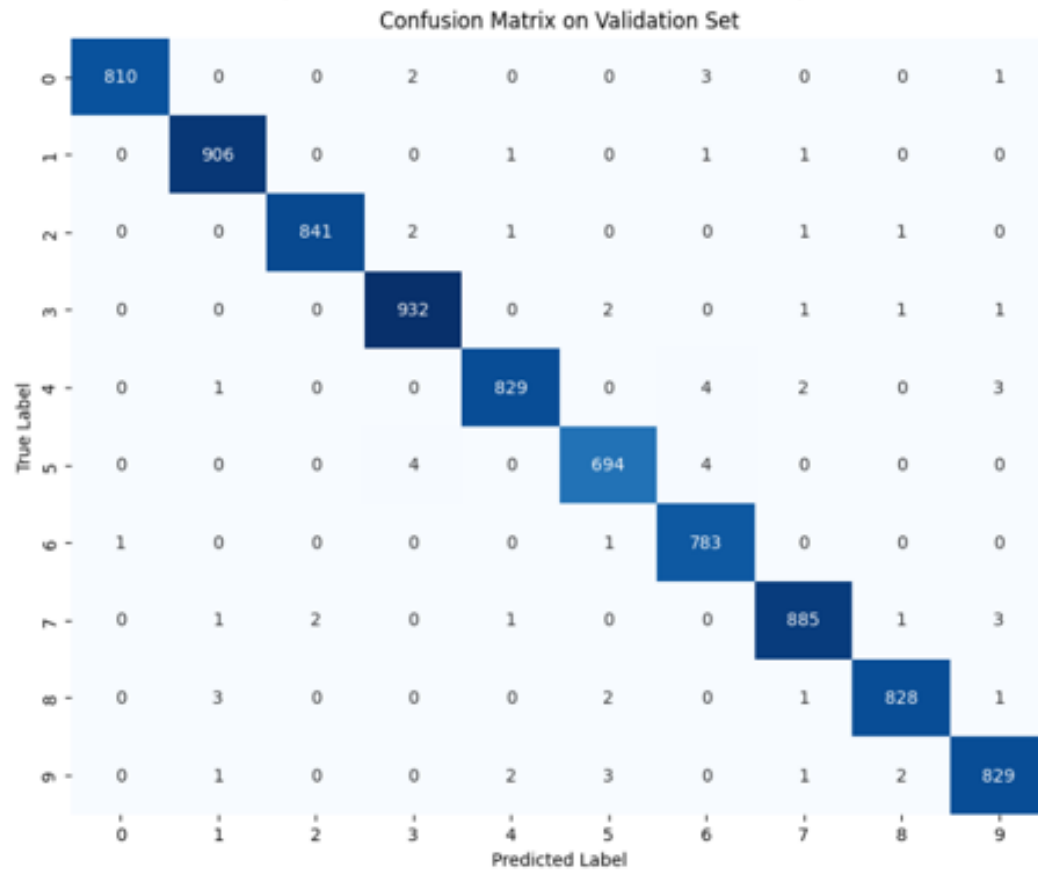
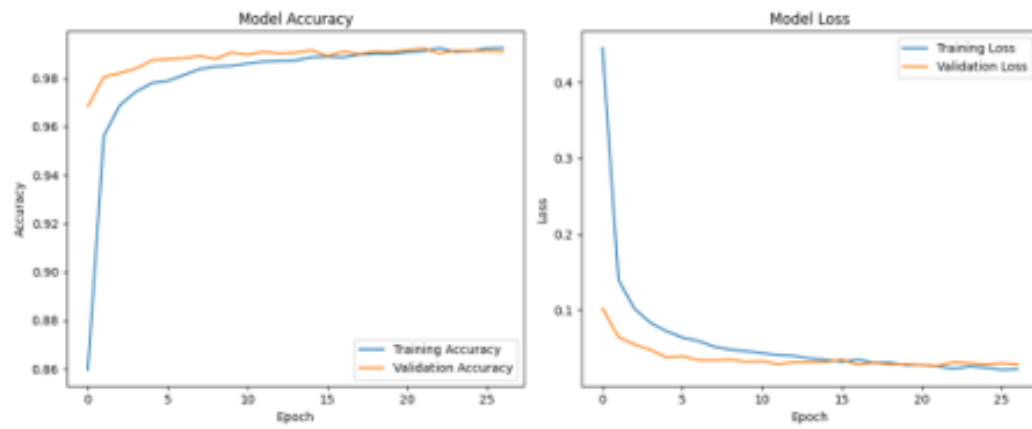
```

Deep Learning

Lampiran 5 Code training model kasus 2 (ML vs DL)

	Metode	Akurasi (Validation Set)	Waktu Training (detik)	Waktu Inferensi (detik)	Jumlah Fitur (Input Model)
0	HOG + SVM	0.961786	36.632073	23.044804	144
1	PCA + Random Forest	0.939762	65.981682	1.080040	154

Machine Learning



Validation Accuracy: 0.9925
Validation Loss: 0.0270

Deep Learning

Lampiran 6 Hasil training model kasus 2 (ML vs DL)

Digit Recognizer
Learn computer vision fundamentals with the famous MNIST data

Overview Data Code Models Discussion Leaderboard Rules Team Submissions

Submissions

Submission and Description Public Score

Submission and Description	Public Score
case2_submission_01.csv Complete · 1h ago · Data Learning · Convolutional Neural Network (CNN)	0.99075
case2_submission_01.csv Complete · 2h ago · Machine Learning · HOG · SVM	0.95521

Lampiran 7 Submit hasil ke kaggle (Kasus 2)

Digit Recognizer

Overview Data Code Models Discussion Leaderboard Rules Team Submissions

Rank	Participant	Score	Time
547	Lucas catem vianna	0.99085	1 12d
548	Muhammed Salah Malleh	0.99082	6 2mo
549	qwert32321023	0.99082	1 1mo
550	Ignacy Kozakiewicz	0.99078	2 2mo
551	AmanVerma5334	0.99078	1 1mo
552	Rio Ramadhani H	0.99075	2 2m
Your Best Entry! Your most recent submission scored 0.99075, which is an improvement over your previous score of 0.95521. Great job!			
553	Chungho	0.99071	1 1mo
554	MartynasCiligas	0.99071	1 1mo
555	TU NGUYEN ANH	0.99071	4 3d
556	NHANGDI	0.99067	9 1d
557	cgitk	0.99067	1 1d
558	samuel leonardo barazona arching an	0.99067	1 7d

Lampiran 8 Leaderboard kaggle (Kasus 2)

```

results = [] # Clear the results list before running new evaluations to avoid duplicates

def train_evaluate_model(model, X_train_vec, y_train, X_val_vec, y_val, model_name, vectorizer_type):
    # Training Time
    start_time_train = time.time()
    model.fit(X_train_vec, y_train)
    training_time = time.time() - start_time_train

    # Inference Time
    start_time_inference = time.time()
    y_pred = model.predict(X_val_vec)
    inference_time = time.time() - start_time_inference # in seconds

    f1 = f1_score(y_val, y_pred)

    # Number of parameters (approximation for simplicity)
    num_params = 0
    if hasattr(model, 'coef_'):
        num_params += model.coef_.size # Use .size for total elements in the array
    if hasattr(model, 'intercept_'):
        # intercept_ can be a scalar or an array, so check its size attribute
        num_params += model.intercept_.size if hasattr(model.intercept_, 'size') else 1
    elif isinstance(model, MultinomialNB):
        # MultinomialNB has feature_log_prob_ (n_classes, n_features) and class_log_prior_ (n_classes)
        if hasattr(model, 'feature_log_prob_'):
            num_params += model.feature_log_prob_.size
        if hasattr(model, 'class_log_prior_'):
            num_params += model.class_log_prior_.size

    # Memory Usage Approximation (in MB)
    model_memory_bytes = 0
    if hasattr(model, 'coef_'):
        model_memory_bytes += model.coef_.nbytes
    if hasattr(model, 'intercept_'):
        model_memory_bytes += model.intercept_.nbytes # nbytes is for numpy arrays
    elif isinstance(model, MultinomialNB):
        if hasattr(model, 'feature_log_prob_'):
            model_memory_bytes += model.feature_log_prob_.nbytes
        if hasattr(model, 'class_log_prior_'):
            model_memory_bytes += model.class_log_prior_.nbytes
    # Convert bytes to MB. Fallback to sys.getsizeof for overall object size if no specific coef/intercept bytes are found
    model_memory_mb = model_memory_bytes / (1024 * 1024) if model_memory_bytes > 0 else sys.getsizeof(model) / (1024 * 1024)

    results.append({
        'Model': model_name,
        'Vectorizer': vectorizer_type,
        'F1-Score': f1,
        'Training Time (s)': training_time,
        'Inference Time (s)': inference_time,
        'Number of Parameters': num_params,
        'Memory (MB)': model_memory_mb
    })
    print(f"{model_name} with {vectorizer_type} - F1-Score: {f1:.4f}, Training Time: {training_time:.2f}s, Inference Time: {inference_time:.4f}s, Memory: {model_memory_mb:.2f}MB")

# Logistic Regression
train_evaluate_model(LogisticRegression(random_state=42, solver='liblinear'), X_train_tfidf_uni, y_train, X_val_tfidf_uni, y_val, 'Logistic Regression', 'TF-IDF Unigram')
train_evaluate_model(LogisticRegression(random_state=42, solver='liblinear'), X_train_tfidf_bi, y_train, X_val_tfidf_bi, y_val, 'Logistic Regression', 'TF-IDF Bigram')

# Multinomial Naive Bayes
train_evaluate_model(MultinomialNB(), X_train_tfidf_uni, y_train, X_val_tfidf_uni, y_val, 'Multinomial Naive Bayes', 'TF-IDF Unigram')
train_evaluate_model(MultinomialNB(), X_train_tfidf_bi, y_train, X_val_tfidf_bi, y_val, 'Multinomial Naive Bayes', 'TF-IDF Bigram')

# Linear SVM
train_evaluate_model(LinearSVC(random_state=42), X_train_tfidf_uni, y_train, X_val_tfidf_uni, y_val, 'Linear SVM', 'TF-IDF Unigram')
train_evaluate_model(LinearSVC(random_state=42), X_train_tfidf_bi, y_train, X_val_tfidf_bi, y_val, 'Linear SVM', 'TF-IDF Bigram')

```

Machine Learning

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout

# Define hyperparameters
embedding_dim = 100 # Dimension of the word embeddings

# Build the LSTM model
model = Sequential([
    Embedding(input_dim=vocab_size, output_dim=embedding_dim), # Removed input_length
    LSTM(128), # LSTM layer with 128 units
    Dense(1, activation='sigmoid') # Output layer for binary classification
])

# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Display model summary
model.summary()

history = model.fit(X_train, y_train,
                    epochs=10, # Number of training epochs
                    batch_size=32, # Batch size for training
                    validation_data=(X_val, y_val))

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Conv1D, GlobalMaxPooling1D, Dense, Dropout

# Define hyperparameters (reusing embedding_dim from previous model)
vocab_size and max_len are also reused from previous preprocessing steps

# Build the 1D-CNN model
model_cnn = Sequential([
    Embedding(input_dim=vocab_size, output_dim=embedding_dim), # Removed input_length
    Dropout(0.5), # Add Dropout for regularization
    Conv1D(filters=32, kernel_size=5, activation='relu'),
    GlobalMaxPooling1D(),
    Dense(64, activation='relu'), # Additional Dense layer
    Dropout(0.5), # Another Dropout layer
    Dense(1, activation='sigmoid') # Output layer for binary classification
])

# Compile the model
model_cnn.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Display model summary
model_cnn.summary()

import time # Added this import

print("Melatih model 1D-CNN dengan Early Stopping...")
start_time_cnn_train = time.time() # Added
history_cnn = model_cnn.fit(X_train, y_train,
                            epochs=50, # Set a higher number of epochs, Early Stopping will manage it
                            batch_size=32,
                            validation_data=(X_val, y_val),
                            callbacks=[early_stopping])

end_time_cnn_train = time.time() # Added
training_time_cnn = end_time_cnn_train - start_time_cnn_train # Added

num_params_cnn = model_cnn.count_params() # Added
memory_mb_cnn = num_params_cnn * 4 / (1024 * 1024) # Assuming float32 parameters, Added

print(f"1D-CNN Training Time: {training_time_cnn:.2f} seconds") # Added
print(f"1D-CNN Number of Parameters: {num_params_cnn}") # Added
print(f"1D-CNN Estimated Memory: {memory_mb_cnn:.2f} MB") # Added

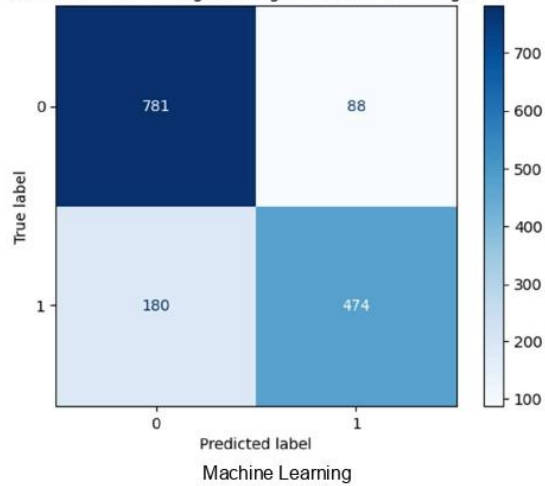
```

Deep Learning

Lampiran 9 Code training model kasus 3 (ML vs DL)

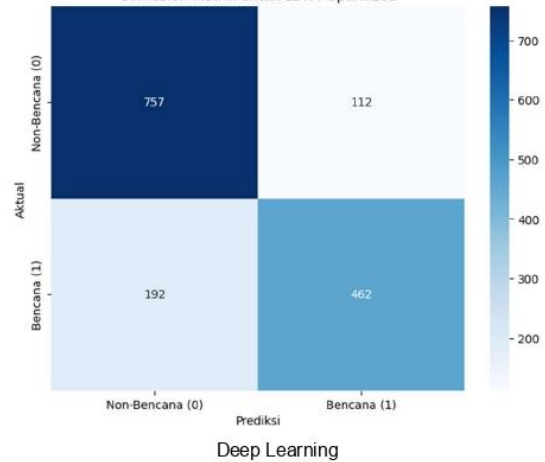
	Model	Vectorizer	F1-Score	Training Time (s)	Inference Time (s)	Number of Parameters	Memory (MB)
0	Logistic Regression	TF-IDF Unigram	0.779605	0.016730	0.000848	5001	0.038155
1	Logistic Regression	TF-IDF Bigram	0.779070	0.017796	0.000831	5001	0.038155
4	Linear SVM	TF-IDF Unigram	0.758077	0.027746	0.001538	5001	0.038155
2	Multinomial Naive Bayes	TF-IDF Unigram	0.754430	0.004070	0.000668	10002	0.076309
3	Multinomial Naive Bayes	TF-IDF Bigram	0.753899	0.003762	0.000619	10002	0.076309
5	Linear SVM	TF-IDF Bigram	0.750988	0.028466	0.000795	5001	0.038155

Confusion Matrix: Logistic Regression (TF-IDF Unigram)



	Model	Vectorizer	F1-Score	Training Time (s)	Inference Time (s)	Number of Parameters	Memory (MB)
0	LSTM Optimized	Keras Tokenizer + Embedding	0.752443	32.780209	2.645803	1552121	5.920872
1	1D-CNN	Keras Tokenizer + Embedding	0.742044	30.772456	0.434966	1457849	5.561253

Confusion Matrix untuk LSTM Optimized



Lampiran 10 Hasil training model kasus 3 (ML vs DL)

Natural Language Processing with Disaster Tweets
Predict which Tweets are about real disasters and which ones are not

Overview Data Code Models Discussion Leaderboard Rules Team Submissions

Submissions

Submission and Description Public Score

Submission and Description	Public Score
case3_submission_01.csv Complete · New · Deep Learning · Embedding + Long Short-Term Memory (LSTM)	0.78271
case3_submission_01.csv Complete · 2h ago · Machine Learning · Logistic Regression + TF-IDF Uprgram	0.79088

Lampiran 11 Submit hasil ke kaggle (Kasus 3)

Natural Language Processing with Disaster Tweets

Overview Data Code Models Discussion Leaderboard Rules Team Submissions

Rank	Name	Score	Rank	Time
437	Bob Kelly	0.79160	4	1mo
438	Mohammed ALLEZ	0.79129	2	10d
439	Anay Gm	0.79098	1	2mo
440	Jiale Lin	0.79098	1	1mo
441	SUHANI THAKARE	0.79098	1	8d
442	NajiM	0.79098	2	8h
443	Riza Ramadhani H	0.79098	2	1m
Your Best Entry! Your most recent submission scored 0.78271, which is not an improvement of your previous score. Keep trying!				
444	PedroFerreir02	0.79068	1	1mo
445	Raj Sheel Gautam	0.79068	2	1mo
446	Fauzdanik8133	0.79068	1	1mo
447	Huu Canh Nguyen	0.79068	1	2d
448	Muzz0rH	0.79037	1	2mo
449	Q.20198	0.79037	6	2mo

Lampiran 12 Leaderboard kaggle (Kasus 3)